

The Problem We're Solving

Every time we build a new software feature, someone has to manually write tests to make sure it works.

What that looks like today:

- A QA engineer reads through requirement documents — takes **30–60 minutes** just to understand scope
- Then writes test scripts by hand — another **2–4 hours per feature**
- If requirements change, tests need to be rewritten from scratch
- Results depend heavily on **individual knowledge** — easy to miss edge cases

Bottom line: QA is a bottleneck. It slows down releases and still misses bugs.

Our Solution — AgenticQAEngine

Describe a feature in plain English → AI handles the rest

Step	What Happens	Who Does It
1. Describe	Type what the feature should do	You (1 sentence)
2. Generate	AI reads our requirements and writes test scenarios	AI (2 min)
3. Review	QA engineer checks and approves the scenarios	QA (5 min)
4. Build Tests	AI writes the actual test code	AI (1 min)
5. Run	Tests execute automatically against the application	System (5 min)
6. Fix	AI analyses failures and suggests fixes	AI + QA (5 min)

Total time: What used to take a full day now takes under 20 minutes.

How the AI Knows What to Test

It doesn't guess — it reads our actual requirements

Knowledge Sources the AI Uses:

Confluence — Our internal requirement pages and test plans are automatically loaded into the system

Neo4j Knowledge Graph — 171 structured requirement nodes across all OrangeHRM modules (Login, Dashboard, Recruitment, Employee Management, Admin)

Domain Guard — The AI only works within our defined scope. Off-topic requests are blocked automatically with a clear message

The AI never makes up requirements. Every test scenario it generates is **grounded in our actual documentation**.

Human Control at Every Step

AI suggests — humans decide

```
[AI Generates Scenarios]
|
v
Gate 1 - QA Reviews Scenarios
Approve / Give Feedback / Upload Own Version
|
v
[AI Writes Test Code]
|
v
Gate 2 - QA Reviews Code
Edit Directly / Request Changes / Approve & Save
|
v
[Tests Run Against Application]
|
v
Gate 3 - QA Reviews Failures
AI Explains Root Cause / One-Click Fix / Mark as Bug
```

Nothing is saved or run without explicit human approval.

What Happens When Tests Fail

The AI doesn't just report failures — it explains and fixes them

AI Failure Analysis (Gate 3):

Failure Type	What It Means	AI Action
App Bug	Feature is broken in the application	Flags it — no code change
Locator Drift	A button or field was renamed in the UI	Proposes updated selector
Wrong Assertion	AI wrote an incorrect expected value	Proposes corrected check
Timing Issue	Page loaded too slowly during the test	Proposes a wait adjustment

- Engineer clicks **Apply Fix** → code is patched instantly
- Then clicks **Re-run This Test** → confirms the fix worked
- If still failing → AI re-analyses automatically with a new suggestion

Live Demo

Let's see it in action

Feature: User Login & Authentication — OrangeHRM

Demo Recap — What You Just Saw

Step 1 — One sentence in: "Generate test cases for user login and authentication"

Step 2 — AI output in ~2 min: 26 test scenarios covering valid login, invalid password, empty fields, locked accounts, session expiry

Step 3 — Human feedback: "Add more edge cases for locked accounts" → AI regenerated in ~20 seconds (resumes from where it left off, skips repeat work)

Step 4 — Test code generated: Full Playwright test suite, ready to run

Step 5 — Failure triage: AI identified a locator change, proposed a fix, test passed on re-run

Impact & Time Savings

Activity	Before (Manual)	After (AgenticQAEEngine)	Saved
Understanding requirements	30–60 min	0 min (AI reads docs)	~45 min
Writing test scenarios	2–3 hours	2 min (AI generates)	~2.5 hrs
Writing test code	3–5 hours	1 min (AI codes)	~4 hrs
Debugging failures	1–2 hours	5 min (AI diagnoses)	~1.5 hrs

Total per feature: 6–10 hours → under 20 minutes | Consistency guaranteed | Zero knowledge dependency

What We've Built So Far

Current system covers the OrangeHRM HR platform

Modules with full test coverage:

- **Authentication & Authorization** — login, logout, session, password policies
- **Admin Management** — user creation, role assignment, permissions
- **Recruitment** — candidate pipeline, job postings, interview workflow
- **Employee Management (PIM)** — employee records, personal info, reporting
- **Dashboard** — navigation, access control, role-based views

Test Suites Generated & Saved:

- Multiple suites already running in CI
- Failure artifacts (screenshots + DOM snapshots) captured automatically
- AI triage available on any failing test with one click

What's Next — Roadmap

Priority	Initiative	Benefit
Short-term	Expand to more modules — Payroll, Leave, Time & Attendance	Wider safety net
Short-term	CI/CD integration — run suites automatically on every code push	Zero manual trigger needed
Medium-term	Cross-domain RAG — connect to other internal systems beyond OrangeHRM	Reuse across projects
Medium-term	Parallel test execution — run all test suites simultaneously	5× faster results
Long-term	Video capture — record browser session during test runs for debugging	Visual evidence per failure
Long-term	Auto-update tests on requirement change — AI detects doc changes and flags outdated tests	Always in sync

Summary

One sentence in → production-ready tests out

- AI reads our requirements → generates scenarios → writes code → runs tests → diagnoses failures
- QA engineer stays in control at every gate — approves, edits, or rejects
- 6–10 hours of manual work reduced to under 20 minutes per feature
- No knowledge dependency — any team member can trigger a test suite
- Built on OrangeHRM today — architecture ready to scale

Thank You — Q&A

Key Takeaways:

Speed: 6–10 hours of QA work → under 20 minutes per feature

Quality: AI covers all edge cases, every time — no Friday-afternoon misses

Control: 3 human approval gates — nothing runs without sign-off

Scale: Same system, any module, any future platform

"We didn't remove QA engineers from the loop — we removed the repetitive parts of their job."

